VERIQTA

ENGINEERING NOTEBOOK SERIES

PROJECT: _____

ENGINEER: _____

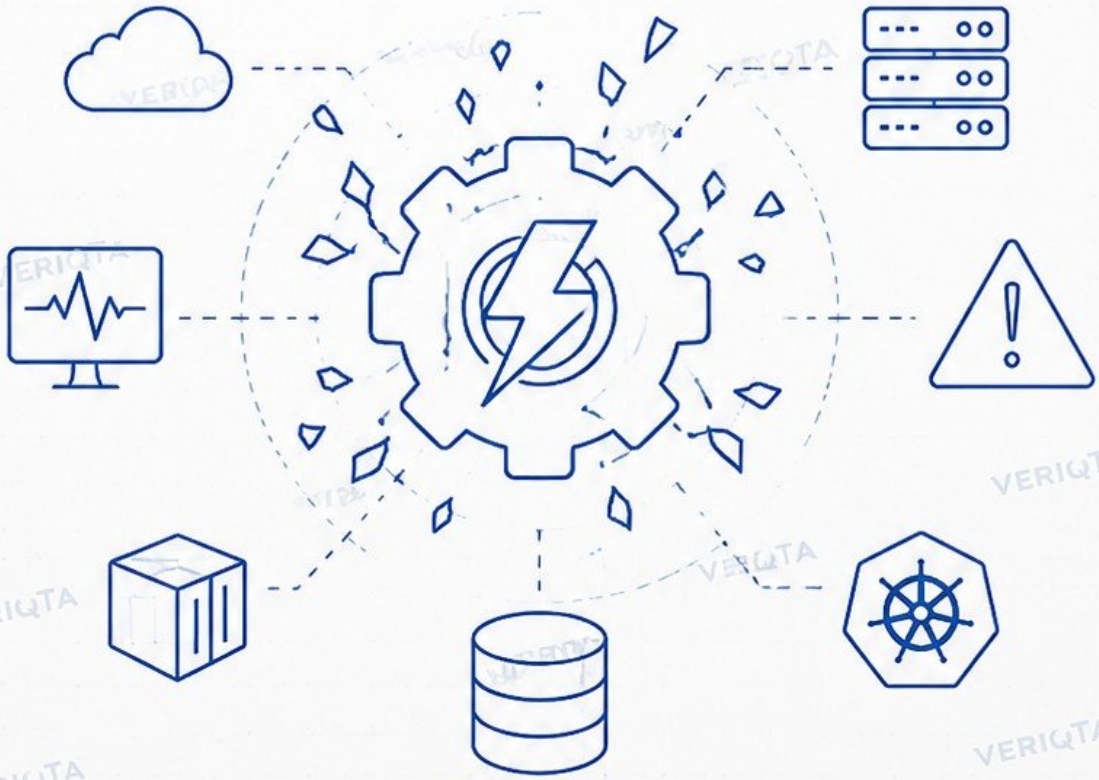
DATE: _____

VERSION: _____

CHAOS ENGINEERING

HANDBOOK

BUILD RESILIENT SYSTEMS.
EXPERIMENT WITH FAILURE. IMPROVE RELIABILITY.



YouTube @veriqta | GitHub @veriqta | LinkedIn @veriqta | X @veriqta | Instagram @veriqta | Telegram @veriqta

VERIQTA

ENGINEERING NOTEBOOK SERIES

PROJECT: _____

ENGINEER: _____

DATE: _____

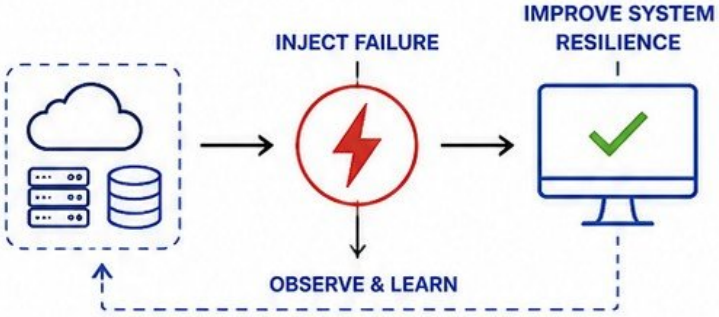
VERSION: _____

CHAOS ENGINEERING FUNDAMENTALS

BUILD RESILIENT SYSTEMS. EXPERIMENT WITH FAILURE.

1 WHAT CHAOS ENGINEERING IS

Chaos engineering is the practice of running controlled experiments in production to find weaknesses before they cause outages. It helps teams understand how their systems behave under real-world failures and improve resilience.



2 RELIABILITY VERSUS RESILIENCE

RELIABILITY	RESILIENCE
• Prevents failures • Focus on uptime • Build systems that don't fail • Measured by availability	• Withstands and recovers from failures • Focus on recovery • Build systems that recover fast • Measured by recovery capability

Reliable systems try to avoid failure. Resilient systems assume failure will happen and are ready to recover.

3 CONTROLLED FAILURE EXPERIMENTATION

- Hypothesis-driven experiments
- Run in production (carefully)
- Small blast radius
- Observe real behavior
- Learn and improve
- Repeat continuously

4 CHAOS ENGINEERING VERSUS ORDINARY TESTING

ORDINARY TESTING	CHAOS ENGINEERING
Runs in pre-production	Runs in production
Predictable scenarios	Unpredictable, real-world failures
Confirms expected behavior	Reveals unknown weaknesses
Finds bugs before release	Finds gaps after release
Stop when tests pass	Continuous learning
Focused on code quality	Focused on system resilience

Testing answers: "Does it work?"
Chaos engineering answers: "Will it still work when something breaks?"

5 PRODUCTION ENGINEERING MINDSET

EMBRACE FAILURE
Assume failure will happen. Design for it.

BE CURIOUS
Always ask: "What can go wrong?"

MEASURE EVERYTHING
You can't improve what you don't measure.

LEARN CONTINUOUSLY
Every experiment should make the system better.

FOCUS ON IMPACT
Protect users. Minimize blast radius. Recover fast.

YouTube @veriqta | GitHub @veriqta | LinkedIn @veriqta | X @veriqta | Instagram @veriqta | Telegram @veriqta

VERIQTA

ENGINEERING NOTEBOOK SERIES

PRINCIPLES OF CHAOS ENGINEERING

BUILD RESILIENT SYSTEMS. EXPERIMENT WITH FAILURE.

1 STEADY-STATE BEHAVIOR

- Understand how the system behaves under normal conditions.
- Establish a baseline of metrics, logs, and user experience.
- Know what "normal" looks like before introducing chaos.

2 HYPOTHESIS-DRIVEN EXPERIMENTS

- Every experiment starts with a clear hypothesis.
- Define what you expect to happen and why.
- Validate or invalidate the hypothesis with real data.

3 REAL-WORLD FAILURE CONDITIONS

- Recreate the types of failures that occur in production.
- Use realistic failure modes, not edge cases.
- Model the unknowns and the unpredictable.

4 BLAST RADIUS

- Limit the impact of experiments to a specific scope.
- Protect users and critical business functions.
- Start small, isolate, and expand gradually.

5 CONTINUOUS EXPERIMENTATION

- Reliability is not a one-time activity.
- Run experiments regularly and continuously.
- Improve resilience as systems and environments change.

6 LEARNING FROM FAILURE

- Failure is a source of valuable knowledge.
- Capture insights and share them with the team.
- Turn lessons into improvements.

YOUTUBE @veriqta | GITHUB @veriqta | LINKEDIN @veriqta | X @veriqta | INSTAGRAM @veriqta | TELEGRAM @veriqta

VERIQTA

VERIQTA

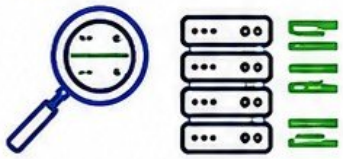
ENGINEERING NOTEBOOK SERIES

GOAL:
Continuously improve system resilience through safe, controlled experiments.

BUILDING A CHAOS ENGINEERING STRATEGY

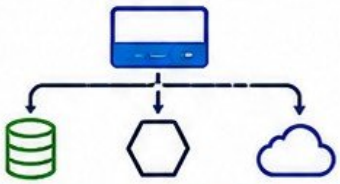
A STRATEGIC APPROACH TO BUILD RESILIENT SYSTEMS.

1 IDENTIFY CRITICAL SERVICES



- List all business services.
- Determine which services are business-critical.
- Focus on services whose failure impacts users or revenue.

2 MAP DEPENDENCIES



- Map service-to-service dependencies.
- Include infrastructure, datastores, and external providers.
- Visualize the dependency graph.

3 DEFINE RELIABILITY EXPECTATIONS

- Define SLOs (e.g., availability, latency, error rate).
- Set error budgets.
- Agree on what "good" looks like under stress.

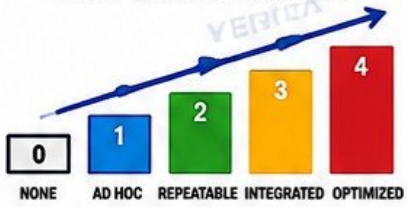
4 CHOOSE EXPERIMENT TARGETS

- Select components whose failure would be meaningful.
- Prioritize high-impact and high-uncertainty areas.
- Start with non-critical paths, then expand.

5 RISK CLASSIFICATION

- Evaluate likelihood and impact.
- Classify risks as Low, Medium, or High.
- Determine required controls for each risk level.

6 EXPERIMENT MATURITY LEVELS



- Level 0: No chaos engineering
- Level 1: Ad hoc experiments
- Level 2: Repeatable processes
- Level 3: Integrated into delivery
- Level 4: Continuous optimization

START SMALL. LEARN FAST.
BUILD RESILIENCE CONTINUOUSLY.

YOUTUBE @veriqta | GITHUB @veriqta | LINKEDIN @veriqta | X @veriqta | INSTAGRAM @veriqta | TELEGRAM @veriqta

VERIQTA

ENGINEERING NOTEBOOK SERIES

SAFE EXPERIMENTS CREATE STRONGER SYSTEMS.

DESIGNING SAFE CHAOS EXPERIMENTS

EVERY GREAT EXPERIMENT STARTS WITH A GREAT DESIGN.

1 EXPERIMENT OBJECTIVE

Define the purpose of the experiment.
Be specific about what you want to learn or validate.

EXAMPLE:
Validate how our checkout service behaves when the payment database becomes unavailable.

2 HYPOTHESIS

State your assumption about how the system will behave.
Make it testable and measurable.

EXAMPLE:
The checkout service will continue processing orders without errors when the database is unavailable.

3 FAILURE INJECTION

Define the failure to be injected.
Specify the target, method, duration, and intensity.

EXAMPLE:
Terminate the primary database instance for 5 minutes using AWS Fault Injection Simulator.

4 EXPECTED BEHAVIOR

Describe how the system should react if it is resilient.
Be clear about acceptable outcomes.

EXAMPLE:
Traffic is automatically routed to the standby database. Users experience no errors or minimal impact.

5 SUCCESS CRITERIA

Define measurable conditions that determine if the experiment is successful.

EXAMPLE:
- Error rate < 1%
- No order loss
- Recovery within 60 seconds

6 ABORT CONDITIONS

Define the signals that require the experiment to stop immediately.
Protect users and critical systems.

EXAMPLE:
- Error rate > 5% for 2 minutes
- High latency > 2 seconds
- Customer impact detected

7 RECOVERY PLAN

Define the steps to restore normal operations after the experiment.
Practice recovery before running.

EXAMPLE:
- Restore primary database
- Verify data integrity
- Monitor until steady state achieved

PLAN IT. TEST IT. LEARN FROM IT. REPEAT.
SAFETY + LEARNING = RESILIENCE

YOUTUBE @veriqta | GITHUB @veriqta | LINKEDIN @veriqta | X @veriqta | INSTAGRAM @veriqta | TELEGRAM @veriqta

VERIQTA

ENGINEERING NOTEBOOK SERIES

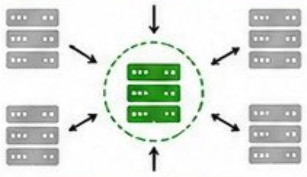
SAFE EXPERIMENTS BUILD STRONG SYSTEMS

BLAST RADIUS AND SAFETY CONTROLS

CONTROL THE CHAOS. PROTECT THE SYSTEM. ENSURE RESILIENCE.

1 LIMITING AFFECTED SYSTEMS

- Define a clear scope for every experiment.
- Target specific components, not entire systems.
- Use tagging, labels, and filters to isolate impact.



2 START SMALL

- Begin with low-intensity, low-risk experiments.
- Validate assumptions before increasing intensity.
- Expand the blast radius only after learning.

3 TEST ENVIRONMENTS VERSUS PRODUCTION

- Test and refine experiments in non-production first.
- Understand differences between environments.
- Use production only when necessary and safe.

4 USER IMPACT LIMITS

- Define acceptable impact on users and business.
- Monitor SLOs, SLIs, and error budgets.
- Abort if impact exceeds defined thresholds.

5 EMERGENCY STOP MECHANISMS

- Implement instant stop capabilities.
- Manual and automated stop options.
- Ensure all teams know how to stop experiments.

6 AUTOMATIC ROLLBACK

- Automatically revert changes when needed.
- Use infrastructure as code and deployment tools.
- Validate system recovery after rollback.

7 CHANGE WINDOWS

- Run experiments during approved time windows.
- Communicate schedules to all stakeholders.
- Avoid high-traffic or critical business periods.

SAFE EXPERIMENTS. SMALL BLAST RADIUS.
BIG LEARNINGS. ZERO REGRETS.

YOUTUBE @veriqta · GITHUB @veriqta · LINKEDIN @veriqta · X @veriqta · INSTAGRAM @veriqta · TELEGRAM @veriqta

VERIQTA

ENGINEERING NOTEBOOK SERIES

BREAK THINGS.
LEARN FAST.
BUILD RESILIENCE.

INFRASTRUCTURE FAILURE EXPERIMENTS

FAILURES WILL HAPPEN. BE READY. TEST. IMPROVE. REPEAT.

1 VM TERMINATION

- Abruptly terminate virtual machines.
- Validate auto-recovery and workload failover.
- Ensure service availability is maintained.

EXAMPLE TOOLS
- AWS FIS, Chaos Monkey
- Azure Chaos Studio
- GCP Fault Injection

2 INSTANCE SHUTDOWN

- Gracefully or forcefully shut down instances.
- Validate service restart and dependencies.
- Ensure minimal impact to users.

EXAMPLE TOOLS
- AWS FIS, SSM Automation
- Azure Chaos Studio
- GCP Fault Injection

3 CPU PRESSURE

- Stress CPU to high utilization.
- Validate auto-scaling and throttling.
- Ensure system remains responsive.

EXAMPLE TOOLS
- Stress-ng, Chaos Mesh
- LitmusChaos
- k6, Locust

4 MEMORY PRESSURE

- Consume memory to create pressure.
- Validate OOM handling and recovery.
- Ensure workloads stay healthy.

EXAMPLE TOOLS
- Stress-ng, Chaos Mesh
- LitmusChaos
- AWS FIS

5 DISK EXHAUSTION

- Fill disk to 100% capacity.
- Validate alerts, graceful degradation.
- Ensure critical services continue.

EXAMPLE TOOLS
- FIO, Stress-ng
- Chaos Mesh
- LitmusChaos

6 DISK I/O DEGRADATION

- Reduce disk read/write performance.
- Validate performance monitoring and alerts.
- Ensure system handles slow I/O gracefully.

EXAMPLE TOOLS
- Toxiproxy (for storage)
- FIO (rate limiting)
- Chaos Mesh

7 HOST FAILURE

- Simulate physical host failure.
- Validate workload rescheduling.
- Ensure high availability is maintained.

EXAMPLE TOOLS
- AWS FIS, Chaos Monkey
- Azure Chaos Studio
- VMware PowerCLI

TEST REAL FAILURES. BUILD RESILIENCE.
STRONG SYSTEMS SURVIVE CHAOS.

YOUTUBE @veriqta · GITHUB @veriqta · LINKEDIN @veriqta · X @veriqta · INSTAGRAM @veriqta · TELEGRAM @veriqta

VERIQTA

ENGINEERING NOTEBOOK SERIES

BREAK THE NETWORK. BUILD RESILIENCE.

NETWORK CHAOS ENGINEERING

CONTROL THE NETWORK CHAOS. STRENGTHEN SYSTEM RESILIENCE.

1 NETWORK LATENCY

- Introduce delay between services.
- Validate user experience and timeouts.
- Ensure SLAs remain within acceptable limits.

EXAMPLE TOOLS

- Toxiproxy
- Chaos Mesh (NetworkChaos)
- AWS FIS (Network Latency)

2 PACKET LOSS

- Drop packets randomly or by percentage.
- Validate retries, error handling, and timeouts.
- Ensure data integrity and user impact limits.

EXAMPLE TOOLS

- Toxiproxy
- Chaos Mesh (NetworkChaos)
- AWS FIS (Packet Loss)

3 NETWORK PARTITION

- Isolate services or zones from each other.
- Validate failover and leader election.
- Ensure system continues operating.

EXAMPLE TOOLS

- Chaos Mesh (NetworkChaos)
- Gremlin
- AWS FIS (Network Partition)

4 BANDWIDTH RESTRICTIONS

- Limit bandwidth in one or both directions.
- Validate performance under constrained links.
- Ensure graceful degradation of services.

EXAMPLE TOOLS

- Toxiproxy
- Chaos Mesh (NetworkChaos)
- Linux tc / netem

5 DNS FAILURES

- Inject DNS timeouts, NXDOMAIN, or failures.
- Validate caching, fallbacks, and retries.
- Ensure services remain discoverable.

EXAMPLE TOOLS

- CoreDNS Chaos Plugin
- Toxiproxy
- AWS FIS (DNS Failure)

6 CONNECTION RESETS

- Reset TCP connections randomly.
- Validate idempotency and retry logic.
- Ensure system handles broken connections.

EXAMPLE TOOLS

- Toxiproxy
- Chaos Mesh (NetworkChaos)
- Gremlin

7 DEPENDENCY ISOLATION

- Isolate or block external dependencies.
- Validate circuit breakers and fallbacks.
- Ensure core functionality remains available.

EXAMPLE TOOLS

- Toxiproxy
- Gremlin
- AWS FIS (Route Block)

TEST THE NETWORK. EXPECT FAILURE.
DESIGN FOR RESILIENCE.

YOUTUBE @veriqta | GITHUB @veriqta | LINKEDIN @veriqta | X @veriqta | INSTAGRAM @veriqta | TELEGRAM @veriqta

VERIQTA

ENGINEERING NOTEBOOK SERIES

RESILIENT APPS DON'T BREAK. THEY RECOVER.

APPLICATION AND SERVICE FAILURES

BREAK THINGS ON PURPOSE. BUILD RESILIENT APPLICATIONS.

1 PROCESS TERMINATION

- Kill application processes or containers.
- Validate auto-restart and self-healing.
- Ensure minimal user impact.

EXAMPLE TOOLS

- Chaos Mesh (Pod/Container Kill)
- AWS FIS (Process Kill)
- Kill -9 / Task Manager

2 SERVICE CRASHES

- Force services to crash unexpectedly.
- Validate restart policies and health checks.
- Ensure failover and recovery work.

EXAMPLE TOOLS

- Chaos Mesh (Container Crash)
- LitmusChaos
- AWS FIS (Crash Instances)

3 DEPENDENCY FAILURES

- Simulate failures of upstream dependencies.
- Validate retries, fallbacks, and circuit breakers.
- Ensure graceful degradation.

EXAMPLE TOOLS

- Toxiproxy
- Chaos Mesh (NetworkChaos)
- Gremlin (Dependency Failure)

4 HTTP ERROR INJECTION

- Inject 4xx/5xx HTTP errors from services.
- Validate error handling and user experience.
- Ensure monitoring and alerts trigger.

EXAMPLE TOOLS

- Toxiproxy
- Istio Fault Injection
- AWS FIS (Return 5xx)

5 APPLICATION LATENCY

- Add delays in application responses.
- Validate timeouts, SLAs, and retries.
- Ensure system responsiveness.

EXAMPLE TOOLS

- Toxiproxy (Latency)
- Chaos Mesh (Delay)
- Gremlin (Latency)

6 THREAD EXHAUSTION

- Exhaust thread pools or worker threads.
- Validate queueing, backpressure, and recovery.
- Ensure system remains stable.

EXAMPLE TOOLS

- Chaos Mesh (Stress)
- JMeter / Locust
- Custom Load Generators

7 CONNECTION POOL EXHAUSTION

- Exhaust database or external connection pools.
- Validate connection timeouts and recovery.
- Ensure new connections can be established.

EXAMPLE TOOLS

- Toxiproxy
- HikariCP Leak Simulation
- Chaos Mesh (Stress)

MAKE FAILURES EXPECTED.
MAKE RECOVERY AUTOMATIC.

YOUTUBE @veriqta · GITHUB @veriqta · LINKEDIN @veriqta · X @veriqta · INSTAGRAM @veriqta · TELEGRAM @veriqta

VERIQTA

VERIQTA

ENGINEERING NOTEBOOK SERIES

RESILIENT CLUSTERS DON'T HAPPEN. THEY ARE TESTED.

KUBERNETES CHAOS ENGINEERING

EXPECT FAILURE. TEST RESILIENCE. BUILD TRUST IN KUBERNETES.

1 POD TERMINATION

- Randomly terminate pods.
- Validate autoscaling and self-healing.
- Ensure minimal user impact.

EXAMPLE TOOLS

- Chaos Mesh (Pod Kill)
- LitmusChaos
- AWS FIS (Kubernetes Pod Kill)

2 CONTAINER FAILURES

- Crash containers inside running pods.
- Validate restart policies and probes.
- Ensure services recover automatically.

EXAMPLE TOOLS

- Chaos Mesh (Container Crash)
- LitmusChaos
- AWS FIS (Container Stop)

3 NODE FAILURES

- Simulate node shutdown or failure.
- Validate pod evictions and rescheduling.
- Ensure workloads remain available.

EXAMPLE TOOLS

- Chaos Mesh (Node Drain)
- LitmusChaos
- AWS FIS (EC2 Stop/Terminate)

4 RESOURCE PRESSURE

- Stress CPU, memory, and disk on nodes.
- Validate limits, requests, and QoS classes.
- Ensure eviction and throttling work.

EXAMPLE TOOLS

- Chaos Mesh (Stress)
- LitmusChaos (CPU/Memory/Disk)
- Kubemark / Kube-burner

5 NETWORK DISRUPTION

- Inject latency, packet loss, or partitions.
- Validate network policies and retries.
- Ensure services handle disruptions.

EXAMPLE TOOLS

- Chaos Mesh (Network Chaos)
- LitmusChaos (Network Chaos)
- Cilium / Toxiproxy

6 DEPLOYMENT FAILURES

- Fail deployments during rollout.
- Validate rollback and surge behavior.
- Ensure configuration errors are handled.

EXAMPLE TOOLS

- Chaos Mesh (Rollout Failure)
- LitmusChaos (App Failure)
- Argo Rollouts (Analysis Failure)

7 KUBERNETES RECOVERY BEHAVIOR

- Observe how Kubernetes detects failures.
- Validate self-healing and reconciliation.
- Ensure cluster returns to steady state.

EXAMPLE TOOLS

- Chaos Mesh (Various Experiments)
- LitmusChaos
- Kube-Observer / KubeEye

TEST YOUR CLUSTER. TRUST YOUR RECOVERY.
CHAOS TODAY, RESILIENCE TOMORROW.

YOUTUBE @veriqta · GITHUB @veriqta · LINKEDIN @veriqta · X @veriqta · INSTAGRAM @veriqta · TELEGRAM @veriqta

VERIQTA

VERIQTA

ENGINEERING NOTEBOOK SERIES

BUILD RESILIENT KUBERNETES. KEEP SERVICES UP.

KUBERNETES RESILIENCE EXPERIMENTS

TEST FAILURE. VALIDATE RECOVERY. STRENGTHEN CLUSTER.

1 REPLICA AVAILABILITY

- Scale down and verify workload survival.
- Test pod replacement.
- Ensure desired replica count.

EXAMPLE TOOLS

- kubectl scale
- Chaos Mesh (Pod Kill)
- Kube-Controller

2 POD RESCHEDULING

- Evict pods from nodes.
- Validate rescheduling.
- Ensure workload continuity.

EXAMPLE TOOLS

- kubectl drain
- Chaos Mesh (Pod Evict)
- Descheduler

3 READINESS AND LIVENESS PROBES

- Break probes.
- Validate failure detection.
- Ensure correct recovery.

EXAMPLE TOOLS

- kubectl get events
- Chaos Mesh (Delay)
- Probe Testing

4 PODDISRUPTIONBUDGETS

- Limit voluntary pod disruptions.
- Validate disruption policies.
- Keep minimum availability.

EXAMPLE TOOLS

- kubectl get pdb
- Chaos Mesh (Pod Kill)
- Kube-PDB Testing

5 REQUESTS AND LIMITS

- Enforce CPU and memory boundaries.
- Test resource pressure.
- Prevent resource exhaustion.

EXAMPLE TOOLS

- kubectl top
- Stress-ng
- Resource Quota Testing

6 AUTOSCALING

- Validate HPA and VPA behavior.
- Test workload growth and shrink.
- Ensure stable scaling.

EXAMPLE TOOLS

- kubectl autoscale
- Metrics Server
- HPA Testing

7 MULTI-NODE RESILIENCE

- Simulate node loss.
- Validate workload rescheduling.
- Ensure cluster stability.

EXAMPLE TOOLS

- kubectl delete node
- Chaos Mesh (Node Kill)
- Node Drain

8 CONTROLLED NODE LOSS

- Shut down nodes safely.
- Validate recovery and healing.
- Test real-world failure.

EXAMPLE TOOLS

- Chaos Mesh (Node Kill)
- kubectl cordon
- kubectl drain

TEST KUBERNETES RESILIENCE.
FAIL FAST. RECOVER STRONG.

VERIQTA

ENGINEERING NOTEBOOK SERIES

CLOUD CHAOS ENGINEERING

CLOUD CHAOS ENGINEERING IS THE PRACTICE OF DELIBERATELY INJECTING FAILURES INTO CLOUD INFRASTRUCTURE AND MANAGED SERVICES TO VALIDATE RESILIENCE, UNCOVER WEAKNESSES, AND STRENGTHEN RECOVERY CAPABILITIES BEFORE REAL OUTAGES HAPPEN.

OBJECTIVE: IMPROVE SYSTEM RESILIENCE, VALIDATE RECOVERY MECHANISMS, PROTECT USER EXPERIENCE, AND INCREASE CONFIDENCE IN CLOUD RELIABILITY.

1 AVAILABILITY ZONE FAILURE

DESCRIPTION:
SIMULATE THE FAILURE OF AN ENTIRE AVAILABILITY ZONE (AZ) WHERE RESOURCES ARE RUNNING.

WHY IT MATTERS:
AZ FAILURES HAPPEN IN THE REAL WORLD. YOUR SYSTEM MUST CONTINUE SERVING TRAFFIC WITH NO MAJOR IMPACT.

WHAT TO VALIDATE:
- MULTI-AZ ARCHITECTURE
- AUTOMATIC FAILOVER
- DATA REPLICATION AND CONSISTENCY
- TRAFFIC ROUTING TO HEALTHY ZONES
- APPLICATION AVAILABILITY (SLOS)

EXAMPLE EXPERIMENT:
- STOP ALL EC2 INSTANCES IN AN AZ
- DISABLE SUBNETS IN A ROUTE TABLE
- BLOCK THE AZ USING NETWORK ACLS
- OBSERVE SYSTEM BEHAVIOR AND RECOVERY

2 INSTANCE LOSS

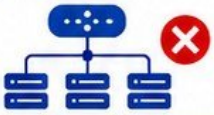
DESCRIPTION:
TERMINATE OR DISRUPT RUNNING INSTANCES (VMS/CONTAINERS/ SERVERLESS WORKERS).

WHY IT MATTERS:
INSTANCES CAN FAIL DUE TO HARDWARE ISSUES, OS CRASHES, KERNEL PANICS, OR CLOUD INTERRUPTIONS.

WHAT TO VALIDATE:
- AUTO SCALING REPLACES INSTANCES
- HEALTH CHECKS DETECT FAILURE
- LOAD BALANCER REMOVES BAD TARGETS
- APPLICATION RECOVERS AUTOMATICALLY
- NO DATA LOSS OR CORRUPTION

EXAMPLE EXPERIMENT:
- TERMINATE RANDOM EC2 INSTANCES
- KILL CONTAINERS OR TASKS
- SIMULATE INSTANCE REBOOT
- OBSERVE REPLACEMENT AND RECOVERY

3 LOAD BALANCER FAILURES



DESCRIPTION:
SIMULATE FAILURE OF LOAD BALANCERS OR THEIR COMPONENTS.

WHY IT MATTERS:
LOAD BALANCERS ARE CRITICAL ENTRY POINTS. THEIR FAILURE SHOULD NOT TAKE DOWN THE APPLICATION.

WHAT TO VALIDATE:
- DNS FAILOVER (ROUTE 53)
- SECONDARY LOAD BALANCER WORKS
- TRAFFIC SHIFTS CORRECTLY
- NO SESSION LOSS (STICKY SESSIONS)
- TLS/SSL CONTINUITY

EXAMPLE EXPERIMENT:
- DELETE OR DISABLE ALB/NLB
- REMOVE TARGETS FROM TARGET GROUP
- CHANGE HEALTH CHECK TO FAIL
- VERIFY TRAFFIC FAILS OVER TO BACKUP

4 STORAGE FAILURES

DESCRIPTION:
SIMULATE FAILURES IN OBJECT STORAGE, BLOCK STORAGE, OR SHARED STORAGE.

WHY IT MATTERS:
STORAGE FAILURES CAN CAUSE DATA UNAVAILABILITY OR APPLICATION ERRORS.

WHAT TO VALIDATE:
- S3 VERSIONING AND CROSS-REGION REPLICATION
- EBS VOLUME REPLACEMENT
- EFS MOUNT TARGET FAILOVER
- DATA DURABILITY AND INTEGRITY
- APPLICATION HANDLES ERRORS

EXAMPLE EXPERIMENT:
- DELETE OBJECTS OR DENY ACCESS
- DETACH EBS VOLUME
- STOP EFS MOUNT TARGETS
- SIMULATE HIGH LATENCY OR I/O ERRORS

5 DATABASE DISRUPTIONS

DESCRIPTION:
INTRODUCE FAILURES OR PERFORMANCE DEGRADATION IN DATABASES.

WHY IT MATTERS:
DATABASES ARE SINGLE POINTS OF FAILURE FOR MANY APPLICATIONS.

WHAT TO VALIDATE:
- READ REPLICAS TAKE OVER
- AUTOMATIC FAILOVER WORKS
- CONNECTION RETRY LOGIC
- TRANSACTION SAFETY
- APPLICATION DEGRADATION BEHAVIOR

EXAMPLE EXPERIMENT:
- FAILOVER PRIMARY DATABASE
- STOP DATABASE INSTANCE
- INCREASE LATENCY
- EXHAUST CONNECTION POOL
- OBSERVE RECOVERY AND DATA SAFETY

6 IAM AND DEPENDENCY FAILURES

DESCRIPTION:
SIMULATE FAILURES IN IAM, SECURITY PERMISSIONS, OR EXTERNAL DEPENDENCIES.

WHY IT MATTERS:
PERMISSION ISSUES OR DEPENDENCY OUTAGES CAN BREAK CRITICAL FLOWS.

WHAT TO VALIDATE:
- IAM ROLE DENIAL HANDLING
- TOKEN EXPIRATION HANDLING
- RETRY AND BACKOFF LOGIC
- CIRCUIT BREAKER BEHAVIOR
- GRACEFUL ERROR HANDLING

EXAMPLE EXPERIMENT:
- REVOKE IAM PERMISSIONS
- EXPIRE CREDENTIALS OR TOKENS
- BLOCK ACCESS TO EXTERNAL APIS
- SIMULATE DNS RESOLUTION FAILURE

7 MANAGED-SERVICE DEGRADATION

DESCRIPTION:
SIMULATE PARTIAL OR COMPLETE DEGRADATION OF MANAGED CLOUD SERVICES (E.G., SQS, SNS, LAMBDA, DYNAMODB, KINESIS).

WHY IT MATTERS:
MANAGED SERVICES CAN THROTTLE, DEGRADE, OR BECOME UNAVAILABLE.

WHAT TO VALIDATE:
- AUTO-RETRIES AND EXPONENTIAL BACKOFF
- DEAD-LETTER QUEUES
- ALTERNATIVE PATHS OR FALLBACKS
- GRACEFUL DEGRADATION
- MONITORING AND ALERTING

EXAMPLE EXPERIMENT:
- THROTTLE DYNAMODB CAPACITY
- DISABLE LAMBDA CONCURRENCY
- STOP SQS CONSUMERS
- INJECT API ERRORS
- VALIDATE FALLBACK + RECOVERY

BEST PRACTICES

START SMALL
Begin with low-risk experiments and increase complexity.

DEFINE BLAST RADIUS
Know what will be affected and limit the impact.

MONITOR EVERYTHING
Use metrics, logs, traces, and alerts during experiments.

AUTOMATE AND REPEAT
Automate experiments and run regularly in production.

LEARN AND IMPROVE
Capture learnings and improve architecture continuously.

YOUTUBE @veriqta | GITHUB @veriqta | LINKEDIN @veriqta | X @veriqta | INSTAGRAM @veriqta | TELEGRAM @veriqta

VERIQTA

ENGINEERING NOTEBOOK SERIES

DATABASE AND STORAGE CHAOS

Database and storage chaos engineering helps you validate how your systems behave under data layer failures. These experiments uncover weaknesses, improve recovery capabilities, and protect data integrity before real outages happen.

OBJECTIVE: VALIDATE DATA LAYER RESILIENCE, ENSURE DATA INTEGRITY, AND STRENGTHEN RECOVERY MECHANISMS BEFORE REAL OUTAGES HAPPEN.

1 DATABASE CONNECTION FAILURE

DESCRIPTION:
Simulate the failure of applications to connect to the database.

WHY IT MATTERS:
Applications must handle connection failures gracefully without data loss or crashes.

WHAT TO VALIDATE:
- Connection retry logic
- Circuit breaker behavior
- Failover to read replica
- Application error handling
- User experience impact

EXAMPLE EXPERIMENT:
- Block database port (3306/5432)
- Revoke security group access
- Stop database proxy
- Disable database endpoint
- Exhaust connection pool

2 INCREASED QUERY LATENCY

DESCRIPTION:
Introduce high latency in database queries.

WHY IT MATTERS:
High latency can degrade performance and lead to timeouts or failures.

WHAT TO VALIDATE:
- Application timeout settings
- Retry and backoff strategies
- Slow query handling
- User experience impact
- Alerting and thresholds

EXAMPLE EXPERIMENT:
- Use Toxiproxy to add latency
- Simulate network latency
- Throttle database resources
- Induce slow queries
- Observe performance metrics

3 REPLICA FAILURE

DESCRIPTION:
Simulate failure of A database replica.

WHY IT MATTERS:
Replicas provide read scaling and high availability. The system must continue operating.

WHAT TO VALIDATE:
- Read traffic failover
- Replica re-election
- Replication lag impact
- Application continuity
- Auto replica recovery

EXAMPLE EXPERIMENT:
- Stop replica instance
- Disable replica endpoint
- Kill replica process
- Simulate replica crash
- Observe failover behavior

4 PRIMARY DATABASE FAILURE

DESCRIPTION:
Simulate failure of the primary (write) database.

WHY IT MATTERS:
The system must failover to a standby without data loss and restore writes quickly.

WHAT TO VALIDATE:
- Automatic failover
- Write availability recovery
- Data consistency
- Application behavior
- Fallback capability

EXAMPLE EXPERIMENT:
- Stop primary database instance
- Detach primary volume
- Simulate datacenter failure
- Promote standby manually
- Observe failover and failback

5 DISK SATURATION

DESCRIPTION:
Consume disk space on the database server or volume until it is full.

WHY IT MATTERS:
Disk full conditions can cause writes to fail and database instability.

WHAT TO VALIDATE:
- Disk full alerts
- Write failure handling
- Database stability
- Log/transaction impact
- Automatic cleanup policies

EXAMPLE EXPERIMENT:
- Fill disk using dummy files
- Generate high log growth
- Induce temp file growth
- Fill volume on cloud disk
- Observe system behavior

6 STORAGE UNAVAILABILITY

DESCRIPTION:
Simulate unavailability of storage volumes used by the database.

WHY IT MATTERS:
Storage failures can make data unreachable and cause service outages.

WHAT TO VALIDATE:
- Attach/detach recovery
- Database restart recovery
- Application failover
- Data access restoration
- Mount point resilience

EXAMPLE EXPERIMENT:
- Detach EBS/volume
- Unmount file system
- Simulate storage controller failure
- Revoke storage permissions
- Observe recovery process

7 BACKUP AND RESTORE VALIDATION

DESCRIPTION:
Validate that backups are usable and restores are successful.

WHY IT MATTERS:
Backups are useless if they cannot be restored when needed.

WHAT TO VALIDATE:
- Backup completeness
- Restore success
- Restore time objectives (RTO)
- Point-in-time recovery
- Automation reliability

EXAMPLE EXPERIMENT:
- Restore to new instance
- Perform point-in-time restore
- Validate backup integrity
- Restore from broken backup
- Test backup automation

8 DATA INTEGRITY SAFEGUARDS

DESCRIPTION:
Validate data consistency and protection mechanisms under failure conditions.

WHY IT MATTERS:
Data integrity is critical for trust, compliance, and business continuity.

WHAT TO VALIDATE:
- Transaction atomicity
- Consistency checks
- Constraint enforcement
- Corruption detection
- Post-recovery validation

EXAMPLE EXPERIMENT:
- Exinterrupt mid-write transaction
- Kill database during write
- Validate checksums/hashes
- Simulate corrupted data blocks
- Verify constraints after recovery

BEST PRACTICES

START SMALL
Begin with low-risk experiments and increase complexity.

DEFINE BLAST RADIUS
Know what will be affected and limit the impact.

MONITOR EVERYTHING
Use metrics, logs, traces, and alerts during experiments.

AUTOMATE AND REPEAT
Automate experiments and run regularly in production-like env.

LEARN AND IMPROVE
Capture learnings and strengthen architecture continuously.

YOUTUBE @veriqta | GITHUB @veriqta | LINKEDIN @veriqta | X @veriqta | INSTAGRAM @veriqta | TELEGRAM @veriqta

VERIQTA

VERIQTA VERIQTA

OBSERVABILITY DURING CHAOS EXPERIMENTS

Observability is what turns chaos into learning.
You can only improve what you can see, measure, and understand.
During chaos experiments, observability helps you detect impact, validate assumptions, and strengthen system resilience.

1 METRICS

- Metrics are numerical measurements that show the behavior and health of your system over time.
- Essential metrics: latency, throughput, error rate, saturation, resource usage, queue depth, request rate.
- Track RED (Rate, Errors, Duration) and USE (Utilization, Saturation, Errors) metrics.
- Use high-resolution metrics to detect fast changes during chaos.
- Compare against baseline to identify deviations and impact.

2 LOGS

- Logs provide detailed, timestamped records of events happening in your system.
- Capture errors, warnings, retries, timeouts, and unusual behavior.
- Use structured logging with consistent fields: timestamp, level, service, trace_id, user_id, error_code, message.
- Centralize logs from all components for easy search and correlation.
- Logs help you understand the why behind metric spikes and failures.

3 TRACES

- Traces show the end-to-end flow of a request across all services.
- Help you see latency breakdowns, bottlenecks, and failure points.
- Every request should have a *trace_id* to follow its full journey.
- Analyze span durations, errors, retries, and service dependencies.
- Traces connect user impact to the exact failing component.

4 INFRASTRUCTURE TELEMETRY

- Infrastructure telemetry covers hosts, containers, networks, and cloud resources.
- Monitor CPU, memory, disk, network I/O, filesystem usage, and process count.
- Track node health, availability, pod status, instance status, and autoscaling events.
- Collect data from cloud providers, Kubernetes, and system exporters.
- Infrastructure telemetry helps identify capacity issues and systemic stress.

5 APPLICATION TELEMETRY

- Application telemetry reflects internal behavior of your application.
- Track business metrics, feature usage, session health, and transaction outcomes.
- Monitor request latency, error rates, retries, fallbacks, and circuit breaker activity.
- Expose custom metrics that matter to your users and business.
- Application telemetry helps measure user experience during chaos.

6 SLO MONITORING

- Service Level Objectives (SLOs) define the reliability targets for your services.
- Monitor SLI indicators: availability, latency, error rate, and saturation.
- Track error budgets and burn rates during experiments.
- Validate whether the system stays within acceptable reliability limits.
- SLO monitoring ensures chaos does not violate user experience.

7 ALERT VALIDATION

- Chaos is a great way to test your alerting system.
- Validate that the right alerts fire, at the right time, with the right severity.
- Check for alert noise, duplication, and missing alerts.
- Confirm alert routing, notifications, and on-call escalation.
- Strong alerts ensure fast detection and fast response during real incidents.

8 CORRELATING FAILURE WITH IMPACT

- The goal is to connect the failure injection to the user impact.
- Use metrics, logs, traces, and telemetry together to build the full story.
- Identify: What failed? Where did it fail? Who was affected? How badly?
- Correlate timelines of failure, detection, alerts, and user impact.
- Document findings to improve architecture, runbooks, and future resilience.

KEY TAKEAWAY

Good observability turns chaos into actionable insights and stronger, more resilient systems.

INJECT FAILURE → OBSERVE IMPACT → CORRELATE THE DATA → IMPROVE RESILIENCE

YouTube @veriqta · GitHub @veriqta · LinkedIn @veriqta · X @veriqta · Instagram @veriqta · Telegram @veriqta

CHAOS ENGINEERING TOOLS

Chaos engineering tools help you safely run controlled failure experiments in your environment to validate resilience, improve reliability, and learn from real-world-like failures.

CHAOS MONKEY

- One of the earliest chaos engineering tools.
- Randomly terminates instances to test application resilience.
- Simple, effective, and widely adopted.
- Best for: EC2 instance failure experiments.

LITMUSCHAOS

- Cloud-native chaos engineering for Kubernetes.
- 300+ ready-to-use chaos experiments.
- Easy to install, configure, and extend.
- Best for: Kubernetes environments.

CHAOS MESH

- Powerful Kubernetes-native chaos engineering platform.
- Supports complex failure scenarios (network, stress, I/O, etc.).
- Workflow-based experiments and rich fault types.
- Best for: Advanced Kubernetes chaos at scale.

GREMLIN

- Commercial chaos engineering platform (SaaS).
- Easy-to-use UI with powerful experiment library.
- Supports infrastructure, network, application, and database chaos.
- Best for: Teams wanting SaaS simplicity and support.

AWS FAULT INJECTION SERVICE

- Fully managed AWS service for running chaos experiments.
- Integrates with CloudWatch, SNS, and other AWS services.
- Built-in guardrails and automatic safety controls.
- Best for: AWS environments.

TOXIPROXY

- Proxy-based tool to simulate network conditions.
- Add latency, bandwidth limits, packet loss, and disconnects.
- Language-agnostic and easy to integrate.
- Best for: Network-related chaos experiments.

TOOL SELECTION CRITERIA

EXPERIMENT REQUIREMENTS

- What types of failures do you need to test?
- Infrastructure, network, application, or all?

ENVIRONMENT COMPATIBILITY

- Kubernetes, cloud provider, on-prem, or hybrid?
- Is the tool supported in your environment?

SAFETY AND GUARDRAILS

- Does the tool provide strong safety controls?
- Can you limit blast radius and define stop conditions?

OBSERVABILITY INTEGRATION

- Does it integrate with your metrics, logs, tracing, and alerting systems?
- Can you measure impact easily?

EASE OF USE & AUTOMATION

- How easy is it to install, configure, and run?
- Can it be used in CI/CD pipelines or GameDays?

COST AND SUPPORT

- Is it open source or commercial?
- Do you need enterprise support?
- What is the total cost of ownership?

COMMUNITY AND MATURITY

- Is the tool widely adopted?
- Is the community active?
- How frequently is it updated?

VERIQTA VERIQTA

AWS FAULT INJECTION SIMULATOR

AWS Fault Injection Simulator (FIS) is a fully managed service that makes it easy to perform controlled experiments that improve the resilience of your applications.

1 EXPERIMENT TEMPLATES

- Pre-built templates for common failure scenarios.
- Customize templates to match your environment.
- Save and reuse templates for consistency.
- Types: AWS provided, Community, or Custom templates.

2 TARGET SELECTION

- Choose the resources that will be impacted.
- Supports tags, resource IDs, or resource groups.
- Granular targeting to control blast radius.
- Preview target resources before running an experiment.

3 IAM PERMISSIONS

- FIS uses an IAM role to perform actions on your behalf.
- Grant least-privilege permissions to the FIS service role.
- Required permissions include: EC2, ECS, EKS, RDS, Lambda, CloudWatch, etc.
- Use separate roles for different environments.

4 STOP CONDITIONS

- Define conditions that automatically stop the experiment.
- Based on CloudWatch alarms or experiment duration.
- Prevents excessive impact and ensures safety.
- Helps control cost and limit blast radius.

5 EC2 EXPERIMENTS

- Actions: stop, terminate, reboot, or isolate instances.
- Simulate instance failures to test auto recovery.
- Target by tags, instance IDs, or ASG.
- Validate HA and failover mechanisms.

6 ECS AND EKS EXPERIMENTS

- ECS: stop tasks, drain instances, or disrupt services.
- EKS: delete pods, disrupt nodes, or network chaos.
- Validate service discovery, rescheduling, and self-healing.
- Test PodDisruptionBudgets, HPA, and failover.

7 NETWORK DISRUPTION

- Actions: latency, packet loss, bandwidth limit, DNS failure.
- Target EC2 instances, containers, or IP ranges.
- Simulate AZ isolation or network partition.
- Validate retries, timeouts, and failover behavior.

8 CLOUDWATCH INTEGRATION

- Monitor experiments with CloudWatch metrics and logs.
- View experiment events and resource metrics.
- Create alarms for stop conditions and notifications.
- Analyze impact and performance in real time.

EXPERIMENT FLOW

CREATE OR SELECT TEMPLATE → SELECT TARGETS → CHOOSE IAM ROLE → DEFINE STOP CONDITIONS → START EXPERIMENT → MONITOR & ANALYZE WITH CLOUDWATCH

YouTube @veriqta | GitHub @veriqta | LinkedIn @veriqta | X @veriqta | Instagram @veriqta | Telegram @veriqta

VERIQTA

VERIQTA VERIQTA

LITMUSCHAOS AND CHAOS MESH

KUBERNETES CHAOS ENGINEERING

LITMUS | CHAOS MESH

LitmusChaos and Chaos Mesh are powerful Kubernetes-native tools used to run controlled chaos experiments to improve the resilience, reliability, and observability of your applications.

1 KUBERNETES INSTALLATION CONCEPTS

- LitmusChaos and Chaos Mesh run inside Kubernetes.
- Ensure you have a working cluster (v1.20+ recommended).
- kubectl is configured to access your cluster.
- You need cluster-admin or sufficient RBAC permissions.
- Helm 3+ is recommended for installation.

CHECK CLUSTER ACCESS

```
$ kubectl get nodes
NAME      STATUS   ROLES           AGE   VERSION
node-1    Ready    control-plane   10d   v1.27.3
```

2 CHAOS EXPERIMENTS

- Chaos experiments introduce failures in a controlled way.
- Define the fault, target, duration, and scope.
- Observe system behavior and validate resilience.
- Always start small and increase blast radius gradually.

COMMON CHAOS ACTIONS

DELAY | LOSS | KILL | STRESS | CORRUPT

3 POD CHAOS

- Target individual pods to test failure scenarios.
- Examples: pod kill, container kill, pod delete, readiness probe failure.
- Validates self-healing, restarts, and availability.

EXAMPLE: POD KILL (LITMUSCHAOS)

```
apiVersion: litmuschaos.io/v1alpha1
kind: PodKill
metadata:
  name: pod-kill
spec:
  duration: "30s"
  target:
    selector:
      labelSelectors:
        app: my-app
```

POD → POD KILLED

4 LITMUSCHAOS & CHAOS MESH INSTALLATION

LITMUSCHAOS (VIA HELM)

```
$ helm repo add litmuschaos https://
charts.litmuschaos.io
$ helm repo update
$ kubectl create ns litmus
$ helm install litmus litmuschaos/
litmus -n litmus --set clusterScoped=true
```

CHAOS MESH (VIA HELM)

```
$ helm repo add chaos-mesh https://
charts.chaos-mesh.org
$ helm repo update
$ kubectl create ns chaos-mesh
$ helm install chaos-mesh chaos-mesh/
chaos-mesh -n chaos-mesh
```

VERIFY INSTALLATION

```
$ kubectl get pods -n litmus
$ kubectl get pods -n chaos-mesh
```

5 NETWORK CHAOS

- Simulate real-world network issues.
- Examples: latency, packet loss, bandwidth loss, DNS failure, network partition.
- Helps validate service-to-service communication and resilience.

EXAMPLE: NETWORK LATENCY (CHAOS MESH)

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: latency-chaos
spec:
  action: delay
  mode: all
  delay:
    latency: "200ms"
    correlation: "50"
  selector:
    namespaces:
      - default
  duration: "60s"
```

POD A → LATENCY 200ms → POD B

6 STRESS CHAOS

- Simulate resource pressure on pods or nodes.
- Examples: CPU stress, memory stress, disk stress.
- Validates autoscaling, resource limits, and stability.

EXAMPLE: CPU STRESS (LITMUSCHAOS)

```
apiVersion: litmuschaos.io/v1alpha1
kind: CPULoad
metadata:
  name: cpu-stress
spec:
  task:
    loadType: cpu
    workers: 2
    load: 80
  duration: "60s"
# Target a specific pod
  target:
    selector:
      labelSelectors:
        app: my-app
```

POD (CPU 80%) — CPU STRESS

7 WORKFLOW EXECUTION

- Chain multiple experiments using workflows.
- Define sequence, parallelism, and dependencies.
- Useful for advanced scenarios and GameDays.

EXAMPLE: WORKFLOW (CHAOS MESH)

```
apiVersion: chaos-mesh.org/v1alpha1
kind: Workflow
metadata:
  name: workflow-sample
spec:
  entry: pod-kill
  templates:
  - name: pod-kill
    templateRef:
      name: pod-kill-template
  - name: network-delay
    templateRef:
      name: network-delay-template
```

STEP 1 POD KILL → STEP 2 NETWORK DELAY

8 EXPERIMENT MONITORING

- Monitor experiments using dashboards and CLI.
- Check experiment status, events, logs, and metrics.
- Integrate with Precurtheus, Grafana, and Alertmanager.

USEFUL COMMANDS

```
# LitmusChaos
$ kubectl get chaosengine
$ kubectl get litmuschaos <name> --sl
$ litmuschaos list

# Chaos Mesh
$ kubectl get chaos
$ kubectl get workflow
$ kubectl describe chaos <name>
```

VIEW LOGS

```
$ kubectl logs -n <namespace> deploy/<tool-deployment>
 --tail=100 -f
```

9 CLEANUP

- Always clean up experiments after execution.
- Prevent leftover resources and unintended impact.
- Delete chaos resources and workflows.

CLEANUP COMMANDS

```
# LitmusChaos
$ kubectl delete chaosengine <name>
$ kubectl delete litmuschaos <name> --all

# Chaos Mesh
$ kubectl delete chaos <name>
$ kubectl delete workflow <name>
```

YouTube @veriqta | GitHub @veriqta | LinkedIn @veriqta | X @veriqta | Instagram @veriqta | Telegram @veriqta

VERIQTA

VERIQTA VERIQTA

CHAOS ENGINEERING IN CI/CD

— BUILDING RESILIENCE INTO EVERY DEPLOYMENT —

Integrate chaos engineering into your CI/CD pipeline to automatically validate resilience, prevent failures in production, and deliver highly reliable systems.

1 AUTOMATED RESILIENCE TESTING

- Automate chaos experiments in CI/CD.
- Run tests at every stage of the pipeline.
- Detect weaknesses early and continuously.
- Reduce manual effort and human error.
- Enforce resilience as a quality standard.

CODE → AUTOMATED CHAOS TESTS → RESILIENCE VALIDATED

2 PRE-PRODUCTION CHAOS

- Run chaos experiments in staging environments.
- Simulate real-world failures before production.
- Validate system behavior under stress.
- Identify gaps before users are affected.
- Build confidence before production release.

DEV → STAGING (CHAOS TESTING) → PRODUCTION (READY)

3 DEPLOYMENT VALIDATION

- Validate deployments under real failure conditions.
- Ensure new versions are resilient.
- Detect regressions in reliability.
- Confirm SLIs and SLOs are maintained.
- Block production if validation fails.

v1.2.3 NEW RELEASE → CHAOS VALIDATION → APPROVED FOR PROD

4 CANARY RELEASES

- Deploy to a small subset of users.
- Run chaos experiments against canary.
- Monitor metrics and user impact closely.
- Promote only if resilience is proven.
- Minimize blast radius and risk.

5% USERS (CANARY) → CHAOS EXPERIMENTS → PROMOTE IF HEALTHY

5 PROGRESSIVE DELIVERY

- Gradually increase traffic to new versions.
- Run chaos at each step of the rollout.
- Continuously evaluate system resilience.
- Pause or rollback if issues are detected.
- Deliver safely with confidence.

10% → 25% → 50% → 100%
TRAFFIC INCREASE + CHAOS VALIDATION AT EACH STEP

6 PIPELINE GATES

- Use chaos results as quality gates.
- Block promotion if resilience criteria fail.
- Define thresholds for SLOs and errors.
- Enforce reliability before each stage.
- Ensure only resilient builds proceed.

CHAOS TEST RESULTS → PASS (PROCEED) ⇢ FAIL (BLOCK)

7 AUTOMATIC ROLLBACK

- Automatically rollback on failure detection.
- Triggered by chaos test failures or alerts.
- Protect users from bad deployments.
- Reduce MTTR and manual intervention.
- Maintain system stability.

DEPLOYMENT → CHAOS TEST FAILS → AUTOMATIC ROLLBACK

8 POST-DEPLOYMENT EXPERIMENTS

- Run chaos in production safely.
- Validate real-world resilience continuously.
- Find hidden weaknesses in live systems.
- Improve reliability after every release.
- Make chaos a continuous practice.

PRODUCTION LIVE → POST-DEPLOYMENT CHAOS → LEARN & IMPROVE

CHAOS ENGINEERING CI/CD PIPELINE OVERVIEW

CODE COMMIT → BUILD → UNIT TESTS → DEPLOY TO STAGING → CHAOS TESTS (STAGING) → VALIDATION GATE → CANARY / PROGRESSIVE DELIVERY → POST-DEPLOYMENT CHAOS → MONITOR & IMPROVE

CONTINUOUS EXPERIMENTATION → CONTINUOUS RESILIENCE → CONTINUOUS DELIVERY

YouTube @veriqta · GitHub @veriqta · LinkedIn @veriqta · X @veriqta · Instagram @veriqta · Telegram @veriqta

VERIQTA

VERIQTA VERIQTA

PRODUCTION CHAOS GAMEDAYS

PRACTICE FAILURE. BUILD RESILIENCE. IMPROVE RELIABILITY

CHAOS GAMEDAYS ARE STRUCTURED, REALISTIC SIMULATIONS OF FAILURES IN PRODUCTION TO TEST PEOPLE, PROCESSES, AND SYSTEMS. THE GOAL IS TO LEARN, IMPROVE, AND BUILD CONFIDENCE.

1 GAMEDAY PLANNING

- DEFINE THE OBJECTIVES.
- CHOOSE THE SCOPE AND BOUNDARIES.
- SELECT THE DATE AND DURATION.
- IDENTIFY SYSTEMS AND DEPENDENCIES.
- GET STAKEHOLDER APPROVAL.
- PREPARE THE TEAM AND RESOURCES.

2 ROLES AND RESPONSIBILITIES

- GAMEDAY LEAD: OVERSEES THE ENTIRE EXERCISE.
- FACILITATOR: GUIDES THE FLOW AND TIMEBOXES.
- ATTACKER: EXECUTES FAILURE SCENARIOS.
- OPERATIONS: RESPONDS AND RESOLVES INCIDENTS.
- OBSERVER: TAKES NOTES AND COLLECTS DATA.
- COMMUNICATION LEAD: MANAGES UPDATES.
- SCRIBE: DOCUMENTS FINDINGS AND TIMELINE.

3 FAILURE SCENARIOS

- BASED ON REAL-WORLD INCIDENTS.
- START SMALL, THEN INCREASE COMPLEXITY.
- EXAMPLES:
  - SERVICE UNAVAILABILITY
  - DATABASE FAILURE
  - NETWORK PARTITION
  - HIGH LATENCY / PACKET LOSS
  - INFRASTRUCTURE OUTAGE
  - THIRD-PARTY DEPENDENCY FAILURE
- DEFINE EXPECTED BLAST RADIUS AND IMPACT.

4 COMMUNICATION CHANNELS

- ESTABLISH PRIMARY COMMUNICATION CHANNEL (CHAT/VOICE).
- CREATE A DEDICATED GAMEDAY CHANNEL.
- DEFINE ESCALATION PATHS.
- SET EXPECTATIONS FOR COMMUNICATION FREQUENCY.
- INFORM STAKEHOLDERS ABOUT TIMELINE AND IMPACT.
- HAVE A BACKUP CHANNEL FOR CRITICAL UPDATES.

5 OBSERVABILITY PREPARATION

- VERIFY METRICS, LOGS, TRACES, AND ALERTS.
- CHECK DASHBOARDS AND INCIDENT DETECTION.
- VALIDATE SLOs AND ERROR BUDGETS.
- ENSURE RUNBOOKS ARE ACCESSIBLE.
- PREPARE DATA COLLECTION FRAMEWORK.
- CONFIRM ALERT ROUTING TO RIGHT PEOPLE.

6 RUNNING THE GAMEDAY

- KICK OFF AND ALIGN EVERYONE.
- START WITH LOW-IMPACT SCENARIOS.
- COMMUNICATE SCENARIO DETAILS TO OPERATIONS.
- ATTACKER INTRODUCES THE FAILURE.
- OPERATIONS DETECTS, TRIAGES, AND RESPONDS.
- FACILITATOR KEEPS THE EXERCISE ON TRACK.
- ADAPT BASED ON TEAM RESPONSE.
- MAINTAIN SAFETY AND BLAST RADIUS.

7 CAPTURING FINDINGS

- DOCUMENT TIMELINE OF EVENTS.
- RECORD DETECTION TIME (MTTD).
- RECORD RESOLUTION TIME (MTTR).
- CAPTURE WHAT WENT WELL.
- IDENTIFY GAPS AND WEAKNESSES.
- COLLECT METRICS AND IMPACT DATA.
- NOTE PROCESS, TOOL, AND PEOPLE ISSUES.
- TAKE ACTIONABLE SCREENSHOTS AND LOGS.

8 POST-EXPERIMENT REVIEW

- DEBRIEF WITH THE ENTIRE TEAM.
- REVIEW OBJECTIVES VS OUTCOMES.
- DISCUSS WHAT WORKED WELL.
- DISCUSS WHAT NEEDS IMPROVEMENT.
- PRIORITIZE ACTION ITEMS.
- ASSIGN OWNERS AND DEADLINES.
- UPDATE RUNBOOKS, ALERTS, AND DOCUMENTATION.
- TRACK AND CLOSE ACTION ITEMS.

GAMEDAY SUCCESS PRINCIPLES

FOCUS ON LEARNING, NOT BLAMING.

IMPROVE SYSTEMS, PROCESSES, AND PEOPLE.

BUILD CONFIDENCE THROUGH PRACTICE.

REPEAT, EVOLVE, AND GET BETTER.

RESILIENCE IS BUILT THROUGH CONTINUOUS EXPERIMENTATION.

VERIQTA

VERIQTA

REAL PRODUCTION CHAOS SCENARIOS

REAL FAILURES. REAL IMPACT. REAL RESILIENCE.

THESE CHAOS SCENARIOS ARE BASED ON REAL-WORLD INCIDENTS THAT HAPPEN IN PRODUCTION. TESTING THEM HELPS YOU BUILD SYSTEMS THAT CAN WITHSTAND THE UNEXPECTED.

1 KUBERNETES NODE DISAPPEARS

- A NODE STOPS RESPONDING SUDDENLY.
- PODS ON THE NODE ARE LOST.
- TESTS SCHEDULING, RESCHEDULING, AND SELF-HEALING.
- VALIDATE PDBs, HPA, AND APPLICATION RESILIENCE.

2 DATABASE BECOMES UNAVAILABLE

- THE PRIMARY DATABASE STOPS RESPONDING.
- CONNECTIONS TIME OUT OR ARE REFUSED.
- TESTS FAILOVER, RECONNECTION, AND RETRY LOGIC.
- VALIDATE RPO, RTO, AND APPLICATION BEHAVIOR.

3 API DEPENDENCY LATENCY INCREASES

- AN EXTERNAL API BECOMES SLOW.
- RESPONSE TIME INCREASES GRADUALLY.
- TESTS TIMEOUTS, RETRIES, CIRCUIT BREAKERS.
- VALIDATE USER EXPERIENCE AND SYSTEM THROUGHPUT.

4 DNS RESOLUTION FAILS

- DNS LOOKUPS START FAILING.
- SERVICES CANNOT RESOLVE DOMAINS.
- TESTS CACHING, FALLBACK DNS, AND RETRIES.
- VALIDATE IMPACT ON SERVICE DISCOVERY AND EXTERNAL CALLS.

5 AVAILABILITY ZONE BECOMES UNAVAILABLE

- AN ENTIRE AZ GOES DOWN.
- RESOURCES IN THAT AZ BECOME UNREACHABLE.
- TESTS MULTI-AZ DEPLOYMENT AND FAILOVER.
- VALIDATE DATA REDUNDANCY, TRAFFIC ROUTING, AND CAPACITY.

6 CPU SATURATION OCCURS

- CPU USAGE SPIKES TO 100%.
- APPLICATIONS SLOW DOWN OR CRASH.
- TESTS RESOURCE LIMITS, THROTTLING, AND AUTO-SCALING.
- VALIDATE ALERTS AND DEGRADATION HANDLING.

7 MESSAGE QUEUE BECOMES UNAVAILABLE

- THE MESSAGE QUEUE STOPS ACCEPTING MESSAGES.
- PRODUCERS FAIL OR BACK UP.
- TESTS RETRIES, DLQ, AND BACKPRESSURE.
- VALIDATE PROCESSING RESILIENCE AND RECOVERY.

8 TRAFFIC SUDDENLY SPIKES

- TRAFFIC INCREASES RAPIDLY.
- SYSTEM RESOURCES ARE STRESSED.
- TESTS AUTO-SCALING, RATE LIMITING, AND QUEUING.
- VALIDATE PERFORMANCE, COST, AND USER EXPERIENCE.

WHY THESE SCENARIOS MATTER

UNCOVER WEAKNESSES BEFORE THEY HAPPEN IN PRODUCTION.

IMPROVE DETECTION, ALERTING, AND OBSERVABILITY.

VALIDATE RECOVERY PROCEDURES AND AUTOMATION.

BUILD TEAM CONFIDENCE THROUGH PRACTICE AND LEARNING.

IMPROVE SYSTEM RESILIENCE AND RELIABILITY.

YouTube @veriqta

GitHub @veriqta

LinkedIn @veriqta

X @veriqta

Instagram @veriqta

Telegram @veriqta

VERIQTA

PRODUCTION CHAOS ENGINEERING PLAYBOOK

VERIQTA

PLAN. EXECUTE. LEARN. IMPROVE. REPEAT.

This playbook gives you a repeatable, safe, and measurable way to run chaos experiments in production and improve resilience.

1 PRE-EXPERIMENT CHECKLIST

- Define the experiment objective.
- Identify the system and scope.
- Validate monitoring and alerts.
- Confirm rollback and recovery plan.
- Stakeholders informed and aligned.
- Select safe time window.
- Verify runbooks are ready.
- Confirm communication channels.

2 HYPOTHESIS TEMPLATE

Hypothesis Statement:

We believe that _____
(justification)

Will result in _____
(expected outcome)

We will know this is true when
(success criteria / metric)

If we are wrong, we expect
(alternative outcome)

3 BLAST-RADIUS CHECKLIST

- What services are in scope?
- What is NOT in scope?
- What users are affected?
- What data is at risk?
- What is the maximum acceptable impact?
- What dependencies are impacted?
- Confirm isolation strategy.

4 ABORT CRITERIA

STOP

- SLO/SLI breach beyond threshold.
- Error rate exceeds limit.
- Latency exceeds limit.
- User impact observed.
- Dependency cascade detected.
- Security risk detected.
- Manual abort triggered by team.

SAFETY FIRST: STOP. ASSESS. RECOVER.

5 EXPERIMENT EXECUTION WORKFLOW

PLAN & PREPARE → RUN EXPERIMENT → OBSERVE & MEASURE → VALIDATE RESULTS

- Start experiment.
- Monitor in real time.
- Compare results with hypothesis.
- Document impact and behavior.
- Decide: Continue / Adjust / Abort.
- End experiment safely.
- Confirm system stability.

6 RECOVERY CHECKLIST

- Stop or revert the chaos.
- Restore affected resources.
- Verify service health.
- Validate dependencies.
- Check dashboards and alerts.
- Confirm SLOs are normal.
- Run smoke tests.
- Ensure no hidden impact.
- Close incident (if any).

7 EVIDENCE COLLECTION

Collect metrics (before, during, after).
Collect logs and traces.

- Capture dashboards screenshots.
- Record timestamps.
- Store experiment events.
- Document impact on users and systems.
- Save alerts and incident data.
- Attach evidence to report.

LOG
TRACE
ALERTS

8 LESSONS LEARNED

- What worked well?
- What didn't work?
- What surprised us?
- What gaps did we find?
- How can we improve?
- What will we automate?

> "Every experiment should make the system better, not just break it."

9 REMEDIATION TRACKING

- Create action items.
- Assign owners.
- Set due dates.
- Track progress.
- Verify fixes.
- Re-test the system.
- Close and document.

TRACK. FIX. VALIDATE. CLOSE THE LOOP.

10 CHAOS MATURITY ROADMAP

LEVEL 1 REACTIVE	LEVEL 2 AWARE	LEVEL 3 STRUCTURED	LEVEL 4 INTEGRATED	LEVEL 5 OPTIMIZED	LEVEL 6 CULTURE
• Incidents drive improvement. • Ad-hoc testing. • Low confidence.	• Basic monitoring. • Simple chaos experiments. • Manual execution.	• Regular experiments. • Defined processes. • Better observability. • Runbooks exist.	• Chaos in CI/CD. • Automated tests. • GameDays. • SLO-driven.	• Continuous learning. • Advanced automation. • Clear ROI. • Strong resilience.	• Resilience culture. • Everyone owns it. • Experiment daily. • Trust the system.

RESILIENCE IS NOT AN ACCIDENT. IT IS BUILT THROUGH INTENTIONAL CHAOS.

YouTube @veriqta | GitHub @veriqta | LinkedIn @veriqta | X @veriqta | Instagram @veriqta | Telegram @veriqta

VERIQTA